Statistical Inference and Modeling

2102575

Suwichaya Suwanwimolkul[1]

[1]Electrical Engineering
Chulalongkorn University

Semester II, 2025

Lectures

- Lecture 1: Introduction
- Lecture 2: Model selection
- Lecture 3: Resampling
- Lecture 4: Linear regression
- Lecture 5: Robust regression
- Lecture 6: Classification
- Lecture 7: Logistic regression
- Lecture 8: Neural networks
- Lecture 9: K-nearest neighbor
- Lecture 10: Bayesian decision, LDA, QDA, and naive Bayes
- Lecture 11: Support vector machine
- Lecture 12: Tree-based methods
- Lecture 13: Bagging, random forecast, boosting
- Lecture 14: Unsupervised learning and clustering algorithms
- Lecture 15: Unsupervised learning and PCA
- Lecture 16: Gaussian mixture model clustering
- Lecture 17: DBSCAN clustering

9. Neural networks

2102575

Suwichaya Suwanwimolkul[1]

[1]Electrical Engineering
Chulalongkorn University

Semester II, 2025

1 Table of Contents

2 Recap: Machine Learning and neural network components

3 Feedforward neural network — the structure

4 The activation functions

5 Configuration of outputs for each task

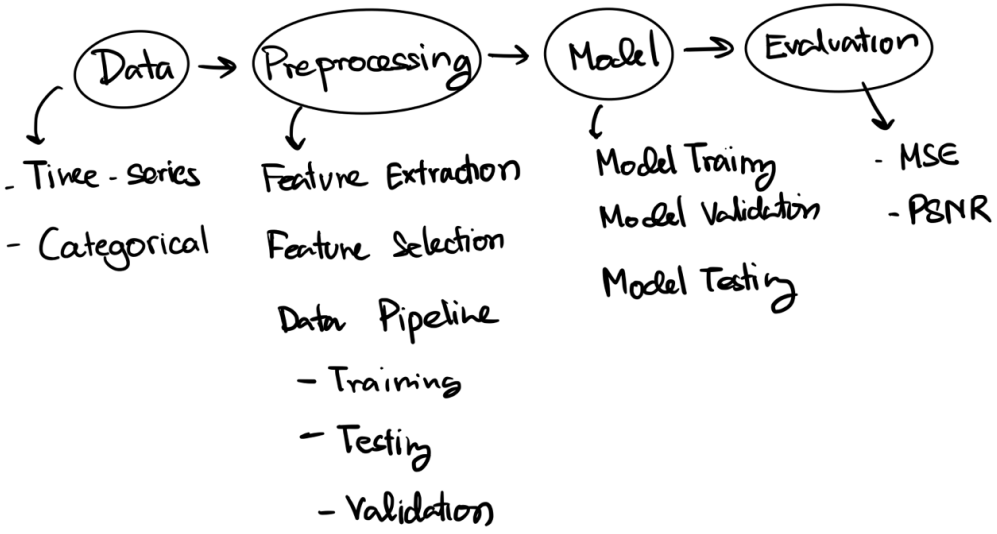
6 Common learning losses
- Regression losses
- Binary classification losses
- Multi-class classification losses

7 Early stopping

8 More examples

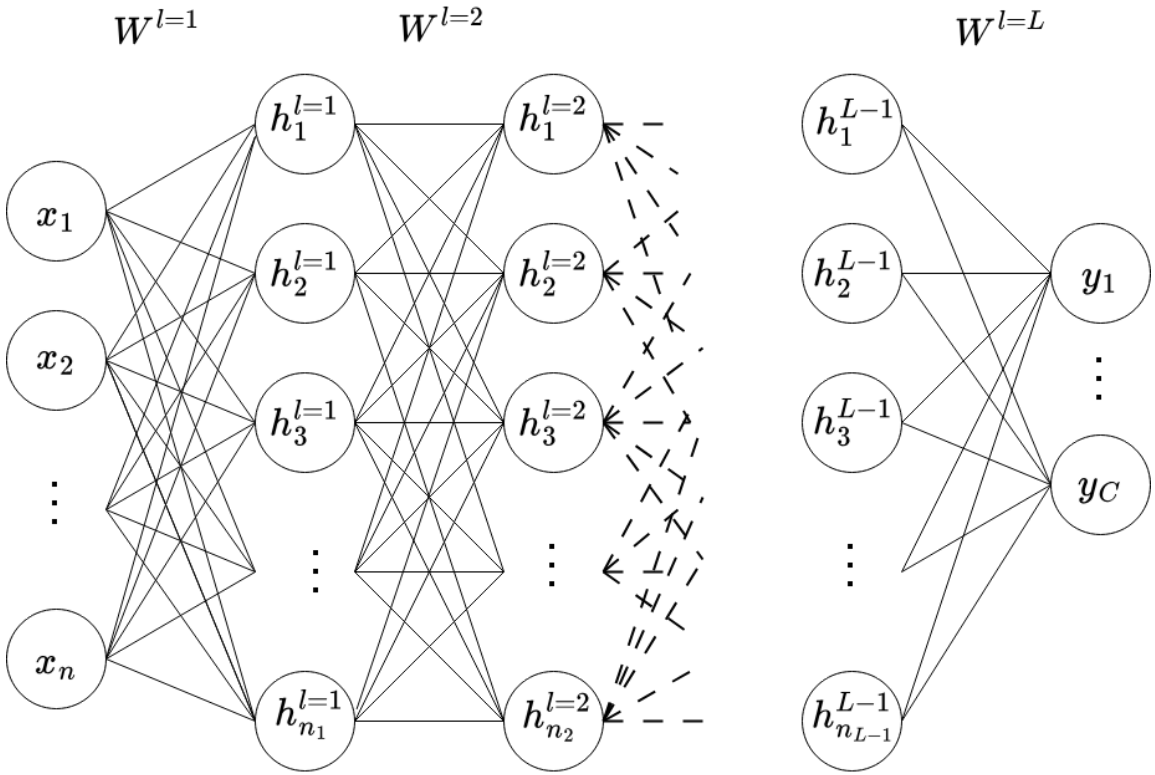
Recap: Machine Learning and neural network components

- Data: x, y.
- Pre-processing
- Model:
  - Linear Regression
  - Robust Regression
  - Regularization
  - Neural networks*
  - Polynomial regression*
  - Spline regression*
- Learning Process.
  - Fitting or Training.
  - Parameter tuning.
  - Feature selection.
- Evaluation



Feedforward neural network — the structure

Fully connected $L-1$ hidden layers; each of which has n_l units resulting from the linear projection with the weight matrix $\boldsymbol{W}^l$ for $l = 1, ..., L-1$.



- $\boldsymbol{x} = [x_1, x_2, ..., x_n]^T$ is the $n-$ dimensional input.
- $\boldsymbol{y} = [y_1, y_2, ..., y_C]^T$ is the $C-$ dimensional output.
- Hidden-layer weight matrices: $\boldsymbol{W}_1 \in R^{n \times n_1} ... \boldsymbol{W}_{L-1} \in R^{n_{L-2} \times n_{L-1}}$
- Output-layer weight: $\boldsymbol{W}_L \in R^{n_L \times C}$

The activation functions

The relationship between the output $\boldsymbol{y}$ and the input $\boldsymbol{x}$, or $\boldsymbol{y} = f(\boldsymbol{x})$, can be described as follows:

$$\begin{aligned}
\boldsymbol{h}^{l=1} &= \boldsymbol{W}^{l=1}\boldsymbol{x} \\
\boldsymbol{h}^{l=2} &= \boldsymbol{W}^{l=2}\boldsymbol{h}^{l=1} \\
\vdots &= \vdots \\
\hat{\boldsymbol{y}} &= \boldsymbol{W}^{L}\boldsymbol{h}^{L-1}
\end{aligned}$$

The above linear model can be extended by adding activation functions as follows:

$$\begin{aligned}
\boldsymbol{h}^{l=1} &= a^{l=1}(\boldsymbol{W}^{l=1}\boldsymbol{x}) \\
\boldsymbol{h}^{l=2} &= a^{l=2}(\boldsymbol{W}^{l=2}\boldsymbol{h}^{l=1}) \\
\vdots &= \vdots \\
\hat{\boldsymbol{y}} &= a^{L}(\boldsymbol{W}^{L}\boldsymbol{h}^{L-1})
\end{aligned}$$

where $a^l(\cdot)$ for $1 \leq l \leq L-1$ are the activation functions at intermediate layers. They are often set to be `Relu`, `MaxPool`, `BatchNorm`, etc, to normalize or scope the range of intermediate outputs.

- The activation function at the output of the neural network $a^{l=L}(\cdot)$, however, is often set to provide the output value at the desired range, *e.g.*, `Relu`, `softmax`, `tanh`, etc.
- **As these activation functions are applied**, the relationship **becomes non-linear!**.

Configuring outputs for each task

Input $\boldsymbol{x} \in \mathcal{R}^{n\times 1}$ is an $n-$ dimensional feature.

- Regression
  - $\hat{\boldsymbol{y}} = f(x)$ where $f(\cdot) : \mathcal{R}^{n\times 1} \Rightarrow \mathcal{R}$
  - *e.g.*

$$\boldsymbol{y} = f(\begin{bmatrix} 0.5 \\ 0.6 \\ 0.7 \end{bmatrix}) = 0.75 \tag{1}$$

- Binary classification
  - $\hat{\boldsymbol{y}} = f(x)$ where $f(\cdot) : \mathcal{R}^{n\times 1} \Rightarrow \mathbb{B}$
  - *e.g.*

$$\hat{\boldsymbol{y}} = f(\begin{bmatrix} 0.5 \\ 0.6 \\ 0.7 \end{bmatrix}) = 0 \text{ or } 1 \tag{2}$$

- Multi-class classification.
  - $\boldsymbol{y} = f(\boldsymbol{x})$ where $f(\cdot) : \mathcal{R}^{n\times 1} \Rightarrow \mathbb{C} = \mathbb{B}^C$
  - C denotes the number of classes.
  - $\mathbb{C}$ is a set of integer correponding to classes, *e.g.*, $\{1, 2, \ldots, C\}$ is mapped to $\{\text{Dog}, \text{ Cat}, \text{ Fish}, \ldots\}$.
  - One can replace $\mathbb{C}$ with a one-hot vector, which is a binary vector $\mathbb{B}^{C\times 1}$ with value '1' at the corresponding class.

$$\hat{\boldsymbol{y}} = f(\begin{bmatrix} 0.1 \\ 0.6 \\ 0.2 \\ 0.1 \end{bmatrix}) = 2 \equiv \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} \text{✗ Dog} \\ \text{✓ Cat} \\ \text{✗ Fish} \\ \text{✗ Tiger} \end{bmatrix}$$

Configuring outputs for each task (cont')

Multi-label and multi-classification

- $\hat{\boldsymbol{y}} = f(\boldsymbol{x})$ where $f(\cdot) : \mathcal{R}^{n\times 1} \Rightarrow \mathbb{C}^L$ or $\mathbb{B}^{L\times C}$.
- L is the number of labels presented in a sample.
- C is the number of classes of a label.
- If $\hat{\boldsymbol{y}} \in \mathbb{C}^L$, $\hat{\boldsymbol{y}}$ will be a vector whose entries correspond to each output label $[\hat{y}_l]_{l=1:L}$.

$$\hat{\boldsymbol{y}} = f(\begin{bmatrix} 0.5 \\ 0.6 \\ 0.7 \end{bmatrix}) = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 2 \end{bmatrix}$$

- If $\hat{\boldsymbol{y}} \in \mathbb{B}^{L\times C}$, $\hat{\boldsymbol{y}}$ will be a matrix where each row entry is the place of label. The column in each row is the class of that label.

$$\hat{\boldsymbol{y}} \equiv \begin{bmatrix} \text{Head :} & \checkmark\text{None, } \times\text{Hat, } \times\text{Beanie} \\ \text{Face :} & \times\text{None, } \checkmark\text{Glass} \\ \text{Body :} & \times\text{None, } \times\text{T-Shirt, } \times\text{Dress, } \checkmark\text{Coat} \\ \text{Feet :} & \times\text{None, } \times\text{Socks, } \checkmark\text{Shoes} \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Note that the output sample has $b = 4$ labels; but each label can have different number of classes, *i.e.*, 3 classes, 2 classes, and 4 classes.

Common learning losses

Goal of learning is about finding the parameters that minimize a chosen loss function.

- where $\mathcal{W} = \{\boldsymbol{W}^l\}$ is the set of all the weights in NN.
- $\{\boldsymbol{x}_j, \boldsymbol{y}_j\}_{j=1}^m$ be m-training data samples containing...
- $\hat{\boldsymbol{y}}_j := f(\boldsymbol{x}_j)$ be the output inferred by the neural network, where $\boldsymbol{x}_j \in \mathcal{R}^{n\times 1}$ denotes the input data/features.

$$\mathcal{W}^* = \underset{\mathcal{W}}{\textbf{argmin}}\ \text{loss}(\boldsymbol{y}_j, \hat{\boldsymbol{y}}_j; \mathcal{W})$$

Learning loss should be chosen in the way that *it supports the task and target output types*.

The target output is $\boldsymbol{y}_j$ of size m, which can be one of the followings:

- Scalar measurement $\hat{\boldsymbol{y}}_j = y \in \mathcal{R}$.
- Binary label $\hat{\boldsymbol{y}}_j = 0/1 \in \mathbb{B} := \{0, 1\}$.
- C-dimensional measurement $\hat{\boldsymbol{y}}_j \in \mathcal{R}^{C\times 1}$; here C is feature dimension size.
- Multi-classes $\hat{\boldsymbol{y}}_j \in \mathbb{B}^{C\times 1}$; here C is the number of classes.
- Multi-classes and multi-label $\hat{\boldsymbol{y}}_j \in \mathbb{B}^{L\times C}$ where L is the number of labels and C is the (maximum) number of classes.

Regression losses

Let $\{\boldsymbol{x}_j, \boldsymbol{y}_j\}_{j=1}^{m}$ are m-training data samples containing...

- the input data/features $\boldsymbol{x}_j \in \mathcal{R}^{n\times 1}$
- the target measurement $\boldsymbol{y}_j \in \mathcal{R}^{C\times 1}$

Regression loss could be one of the followings:

- MSE: $\frac{1}{m}\sum_{j=1}^{m} ||\boldsymbol{y}_j - \hat{\boldsymbol{y}}_j||_2$
- MAE: $\frac{1}{m}\sum_{j=1}^{m} ||\boldsymbol{y}_j - \hat{\boldsymbol{y}}_j||_1$

Binary classification losses

Classification activations: The target label $y_j = \{0, 1\}$ or $\mathbb{B}^{1\times 1}$ (scalar value).

- One can use the NN to infer the probability of the output sample being true, given the input x_j, *i.e.*, $\hat{y}_j = P(y_j = 1|x_j)$.
- $P(y_j = 1|x_j)$ can be computed using `Sigmoid`, `Softmax`, `Relu` etc.
- Activation layers: https://pytorch.org/docs/stable/nn.functional.html

Binary classification losses (I)

Binary classification loss could be one of the followings:

- **Cross entropy loss**:

$$-\sum_{j=1}^{m} y_j \log(\hat{y}_j) + (1 - y_j) \log(1 - \hat{y}_j)$$

- **Hinge loss**:

$$\sum_{j=1}^{m} \max(0, 1 - y_j \cdot \hat{y}_j)$$

  when $y_j \neq \hat{y}_j$, *e.g.*, $\max(0, 1 - 1 \cdot (-1)) = 2$. Here, y_j is $(-1, 1)$, so we have to normalize $\hat{y}_j$ to $(-1, 1)$ as well...

- **F1-score:**

$$\mathsf{F1} = \frac{2\mathsf{TP}}{2\mathsf{TP} + \mathsf{FP} + \mathsf{FN}}$$

  where $TP := \sum_j y_j \hat{y}_j$, $FP := \sum_j (1 - y_j)\hat{y}_j$, and $FN := \sum_j y_j (1 - \hat{y}_j)$.

- For more losses function used in Pytorch Deep Learning: https://lightning.ai/docs/torchmetrics/stable/

Multi-class classification losses

Recap: To formulate the learning, we start from to representing the class of y_j out of C classes with a one-hot vector $\in \{0,1\}^C$.

- For example, the one-hot vector for $y_j = 2$ is $b_{k=2} = [0,1,0,0]$, if there are four classes.
- Previously, we have seen an example like this ... just as a reminder.

$$\begin{bmatrix} \text{✗ Dog} \\ \text{✓ Cat} \\ \text{✗ Fish} \\ \text{✗ Tiger} \end{bmatrix} \equiv \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

- Then, equivalently, we represent $\log p(y_j = 3|x_j)$ as $\log p(b_{k=3}|x_j)$, where $\sum_{k=1}^{C} p(y_j = k|x_j) = \sum_{k=1}^{C} p(b_k|x_j) = 1$.
- $p(b_k|x_j)$ can be calculated as follows:

$$p(b_k|x_j;\beta) = \frac{\exp \beta^T x_j}{1 + \exp \beta^T x_j} \approx \frac{\exp a_k}{\sum_{k=1}^{C} \exp a_k}$$

- Notice $p(b_k|x_j;\beta)$ is a scalar value. In practice, the soft-max layer is used to calculate each $p(b_c|x_j)$ for all $c \in [C]$, that is,

$$\left[p(b_k|x_j;\beta)\right]_{k=1}^{C} = \left[\frac{\exp a_k}{\sum_{c=1}^{C} \exp a_c}\right]_{k=1}^{C}$$

Let's see https://pytorch.org/docs/stable/nn.functional.html

Multi-class classification losses

We can formulate the learning of the model parameter β with a maximum likelihood problem:

$$\underset{\beta}{\textbf{argmax}} \prod_{j=1}^{m} p(b_k|x_j;\beta)$$

where there are m training samples, and each of these samples corresponds to a class $\in \{1, 2, ..., C\}$. b_k is a one hot vector of size C to represent the training sample with kth class of all C classes. One can process the above learning loss in log-scale:

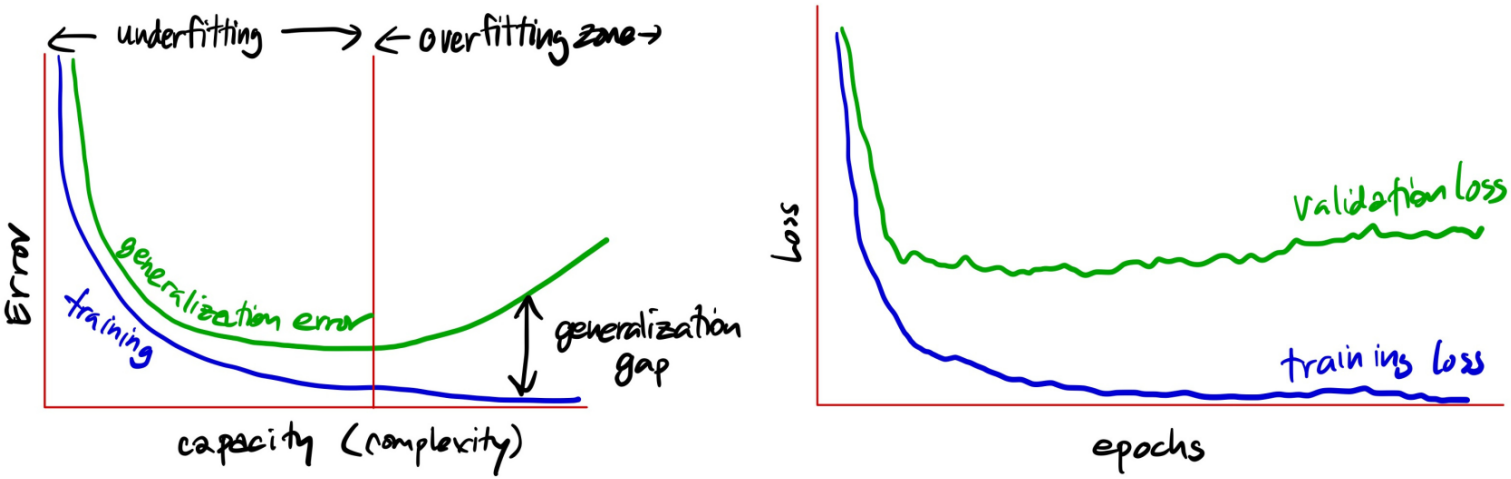
$$\underset{\beta}{\textbf{argmin}} -\sum_{j=1}^{m} \log p(b_k|x_j;\beta)$$

Let $K_1, K_2, ..., K_C$ denotes the sets of training samples correspond to the classes $c = 1$, $c = 2$, ..., $c = C$. Then, the loss function is equivalent to

$$\underset{\beta}{\textbf{argmin}} -\frac{1}{C}\left(\sum_{k1\in K_1} \frac{\exp a_{k1}}{\sum_{c=1}^{C} \exp a_c} + \sum_{k2\in K_2} \frac{\exp a_{k2}}{\sum_{c=1}^{C} \exp a_c} ... + \sum_{kc\in K_C} \frac{\exp a_{kc}}{\sum_{c=1}^{C} \exp a_c}\right)$$

Early stopping

The training loss decreases over time, but validation error may rise again...



Early stopping: to use the solution at the iteration with lowest validation error.

- Run validation error evaluation periodically during training — either in parallel by separate GPU or using small validation set compared to the training set.
- Store the best solution in a separate epoch from training.
- Let's look at an example of classification https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html

More examples

https://github.com/pytorch/tutorials/tree/main/beginner_source/blitz
https://github.com/pytorch/examples/tree/main

References I

[1] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. New York, NY, USA: Springer New York Inc., 2001.

[2] Gareth James et al. *An Introduction to Statistical Learning: with Applications in R*. Springer, 2013. URL: https://faculty.marshall.usc.edu/gareth-james/ISL/.

[3] Jitkomut Songsiri. "Statistical Learning". In: in EE575 Random Processes for EE. Chulalongkorn University, 2022. Chap. 5. Resampling. URL: http://jitkomut.eng.chula.ac.th/ee575.html.